CHAPTER 1

합성곱신경망

성명 : 최승호
이메일 : jcn99250@naver.com
전화번호 : 010-3168-9024

학습 목표

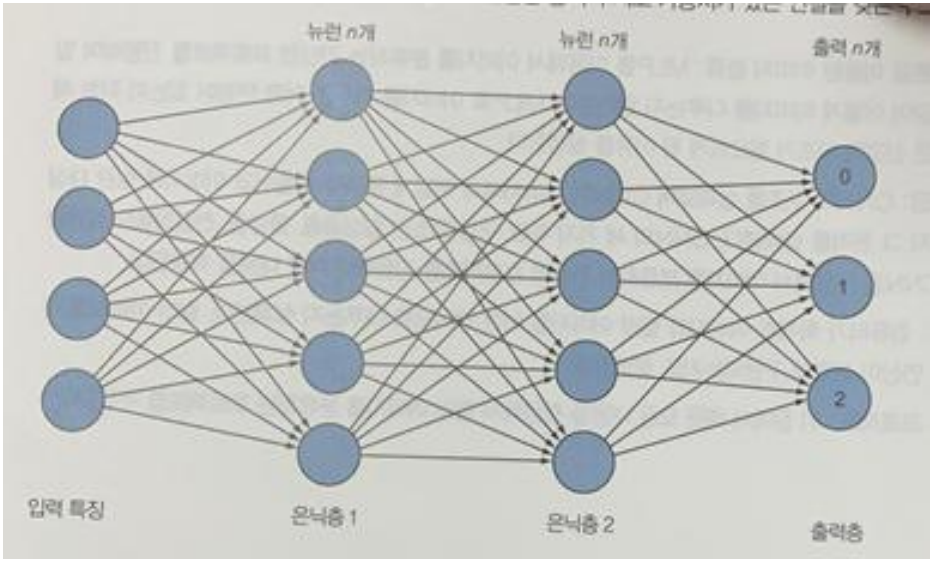
- 다층 퍼셉트론을 이용하여 이미지 분류하기
- 합성 곱 신경망 구조를 이용하여 이미지 분류하기
- 컬러 이미지에 대한 합성 곱 이해하기

기초 지식

- 신경망이 학습하고 파라미터를 최적화 하는 기본 원리는 CNN 과 MLP 모두 동일함
  - 1. 신경망 구조 : MLP 와 CNN은 뉴런층이 겹겹히 쌓인 구조로 구성
  - 2. 가중치와 편향: MLP 와 CNN의 예측은 동일한 방식으로 작동함, 두 가지 모두 무작위 값으로 초기화 되는 가중치와 편향을 갖음, 신경망의 학습 과정에서 이들 값이 학습이 됨
  - 3. 하이퍼파라미터 : MLP 와 마찬가지로 CNN을 설계할 때도 오차 함수, 활성화 함수, 최적화 알고리즘 등을 결정해야 함
  - 4. 학습 : MLP와 CNN학습 방식에 큰차이가 없음, 먼저 순방향 계산을 하고, 예측 결과와 실제 레이블 값을 통해 손실 함수를 계산하고 마지막으로 경사 하강법을 이용해서 파라미터를 최적화하고자 함

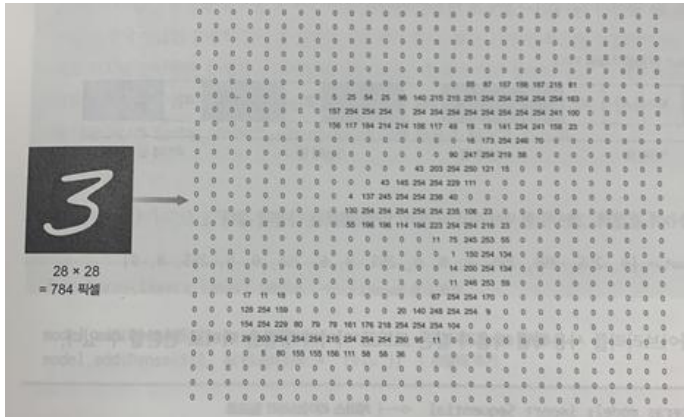
다층 퍼셉트론

- 다층 퍼셉트론 구조
  - 뉴런이 층 모양으로 모여 있고, 인접한 층과 층 사이의 모든 뉴런이 가중치가 있는 연결을 갖음
  - MLP 구조는 하나의 입력층과 하나 또는 그 이상의 은닉층, 하나의 출력층으로 구성됨



다층 퍼셉트론

- 다층 퍼셉트론 구조 입력 층 :
  - 신경망으로 2차원 이미지를 다루려면 이미지를 제공하기 전에 신경망이 이해할 수 있는 형태로 변환해야 함
  - 먼저 컴퓨터는 어떻게 인식하는 지부터 살펴보고자 함
    - 28x28크기의 이미지는 컴퓨터가 보면 28x28크기의 행렬로 구성됨
    - 그리고 이 행렬의 요소 값은 0~255의 범위를 가짐



다층 퍼셉트론

- 다층 퍼셉트론 구조 입력 층 :
  - MLP 는 모양이 (1,n)인 1차원 벡터만 입력 받으므로 모양이 (x,y)인 2차원 이미지 행렬은 입력이 불가능
  - 행렬을 입력층에 입력하려면 행렬을 모양이 (1,n)인 벡터로 변환해야 함, 위 과정을 이미지 벡터 변환 (image flattening)
  - 예를 들어, 앞의 이미지를 예로 들면 픽셀의 총 수(n)는 28x28 = 784 이므로 이 이미지를 신경망에 입력하려면 모양이 (28x28)인 행렬을 모양이 (1x784)인 벡터로 변환해야 함
  - 입력 벡터는 다음과 같이 구성됨 : x=[row1,row2…, row28]
  - 즉, 신경망의 입력층이 가지게 될 노드는 784개가 됨

다층 퍼셉트론

- 다층 퍼셉트론 구조 입력 층 :
  - 케라스에서 구현 내용
    - 소스 코드 중에 Flatten이 2차원 이미지 행렬을 입력 받아 1차원 벡터로 변환하는 과정을 수행함
    - 주의 할 점은 Flatten 층을 만들 때 입력 이미지의 크기를 반드시 파라미터로 지정해야 한다는 점

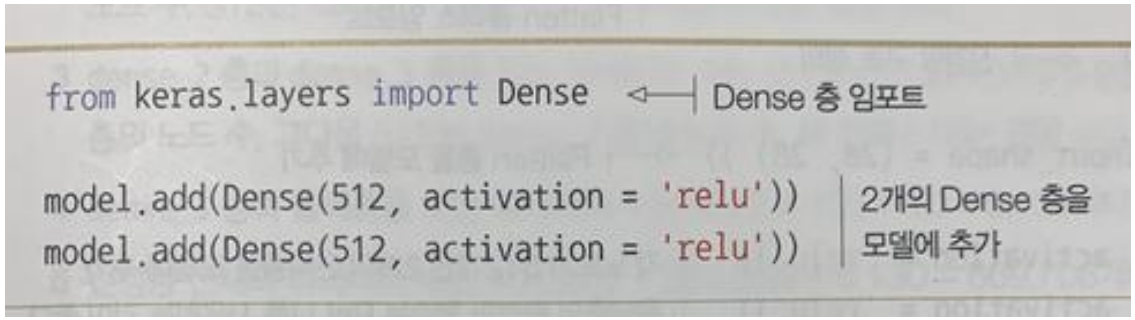
```
from keras.models import Sequential      ← 케라스 라이브러리 임포트
from keras.layers import Flatten         ← 이미지 행렬을 벡터로 변환하는 Flatten 층 임포트

model = Sequential()
model.add( Flatten(input_shape = (28, 28) ))   ← Flatten 층을 입력층으로 모델에 추가한다.
```

모델 정의

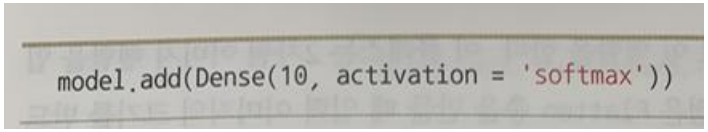
다층 퍼셉트론

- 다층 퍼셉트론 구조 은닉 층:
  - 신경망은 하나 이상의 은닉층을 가질 수 있음 (이론적으로는 원하는 수 만큼 가질 수 있음)
  - 각 층은 하나 이상의 뉴런으로 구성이 됨 (뉴런 수는 원하는 만큼 가능)
  - 모델 설계 시 해야 할 일은 신경망의 층을 설계하는 것
  - 케라스에서 구현시 내용



다층 퍼셉트론

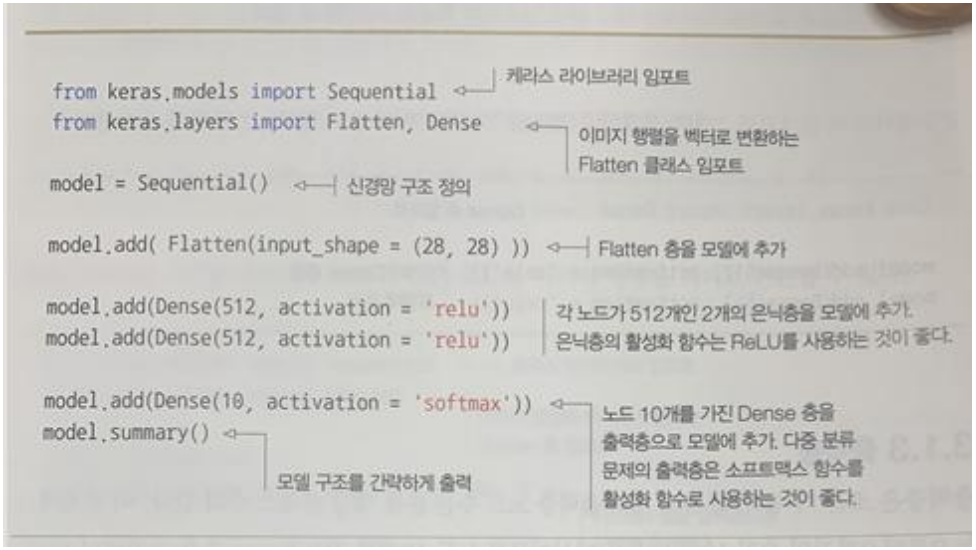
- 다층 퍼셉트론 구조 출력 층:
  - 분류 문제의 출력 층 노드 수는 분류 대상 클래스 수와 같음
  - 위 문제에서는 0부터 9까지의 숫자 10개가 분류 대상이므로 노드 10개를 갖는 Dense layer를 추가하면 됨
  - 케라스에서 구현 시 내용



```
model.add(Dense(10, activation = 'softmax'))
```

다층 퍼셉트론

- 다층 퍼셉트론 전체 구조 :
  - 케라스에서 구현 시 내용



```
from keras.models import Sequential
from keras.layers import Flatten, Dense

model = Sequential()

model.add( Flatten(input_shape = (28, 28)) )

model.add(Dense(512, activation = 'relu'))
model.add(Dense(512, activation = 'relu'))

model.add(Dense(10, activation = 'softmax'))
model.summary()
```

케라스 라이브러리 임포트

이미지 행렬을 벡터로 변환하는 Flatten 클래스 임포트

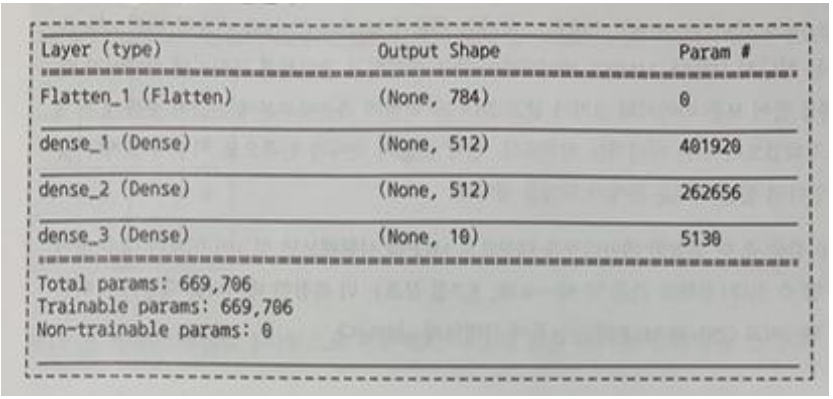
신경망 구조 정의

Flatten 층을 모델에 추가

각 노드가 512개인 2개의 은닉층을 모델에 추가. 은닉층의 활성화 함수는 ReLU를 사용하는 것이 좋다.

노드 10개를 가진 Dense 층을 출력층으로 모델에 추가. 다중 분류 문제의 출력층은 소프트맥스 함수를 활성화 함수로 사용하는 것이 좋다.

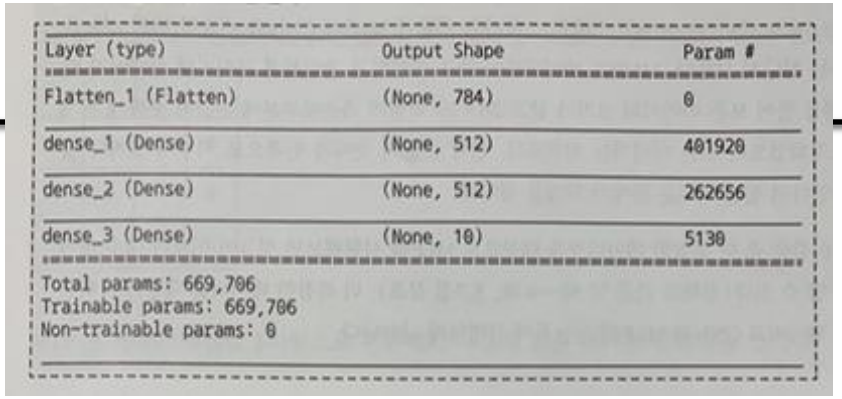
모델 구조를 간략하게 출력



Layer (type)	Output Shape	Param #
Flatten_1 (Flatten)	(None, 784)	0
dense_1 (Dense)	(None, 512)	401920
dense_2 (Dense)	(None, 512)	262656
dense_3 (Dense)	(None, 10)	5130

Total params: 669,706
Trainable params: 669,706
Non-trainable params: 0

다층 퍼셉트론

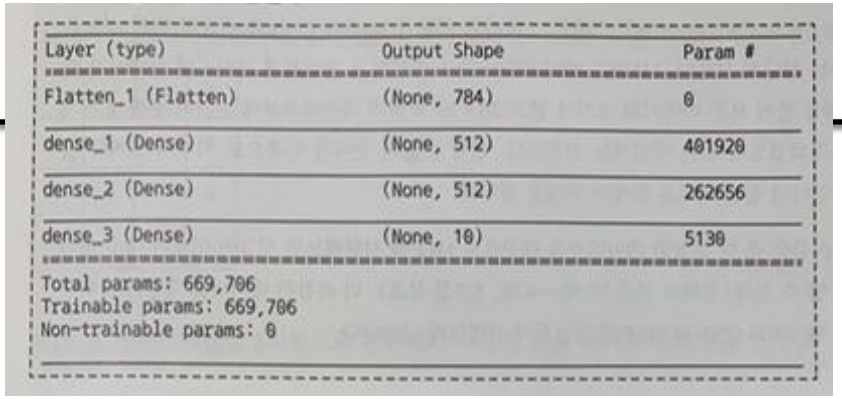


Layer (type)	Output Shape	Param #
Flatten_1 (Flatten)	(None, 784)	0
dense_1 (Dense)	(None, 512)	401920
dense_2 (Dense)	(None, 512)	262656
dense_3 (Dense)	(None, 10)	5130

Total params: 669,706
Trainable params: 669,706
Non-trainable params: 0

- 다층 퍼셉트론 전체 구조 :
  - 출력 부분에 대한 해석:
    - 출력된 내용 중 Param # 필드는 해당 층의 파라미터 (가중치) 수를 의미함
    - 학습은 이들 가중치가 조정되는 과정, 가중치 수는 다음과 같이 계산이 됨
      - 1. Flatten 층은 그 다음 층을 있는 것이기때문에 파라미터 수는 0임, Flatten층은 이미지를 나타내는 행렬을 벡터로 변환하는 역할만 하기 때문
      - 2. Dense_1층과 Dense_2층을 있는 파라미터 수는 (784*512)+512 = 401,920개임, 784는 입력층의 노드 수, 512는 dense_1층의 노드 수, 그 다음 512는 편향수

다층 퍼셉트론



Layer (type)	Output Shape	Param #
Flatten_1 (Flatten)	(None, 784)	0
dense_1 (Dense)	(None, 512)	401920
dense_2 (Dense)	(None, 512)	262656
dense_3 (Dense)	(None, 10)	5130

Total params: 669,706
Trainable params: 669,706
Non-trainable params: 0

- 다층 퍼셉트론 전체 구조 :
  - 출력 부분에 대한 해석:
    - 학습은 이들 가중치가 조정되는 과정, 가중치 수는 다음과 같이 계산이 됨 (계속)
      - 3. Dense_2층과 Dense_3층을 있는 파라미터 수는 (512x512)+512 = 262,656개임, 512는 dense_2 층의 노드 수, 그 다음 512는 dense_3층의 노드 수, 세번째는 512는 편향 수
      - 4. dense_3층과 출력층을 있는 파라미터 수는 (512x10)+10 = 5,130개임
      - 5. 신경망 전체 파라미터 수는 401,920+262,656+5,130=669,706개임

다층 퍼셉트론

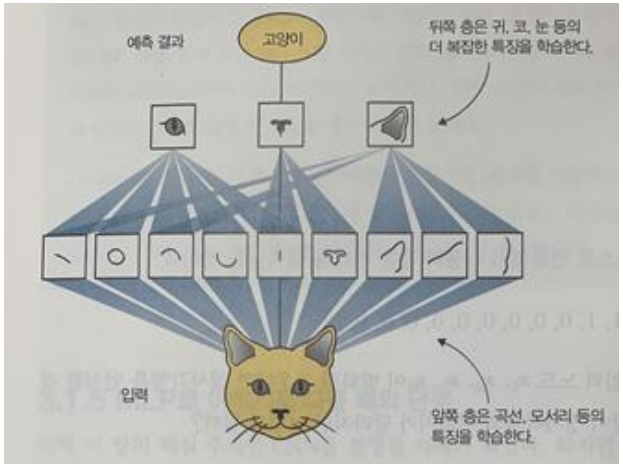
- 다층 퍼셉트론 결론 :
  - 크지 않은 신경망도 무려 669,706개나 되는 파라미터(가중치와 편향)을 통해 학습을 해야 함
  - 신경망의 크기를 감안하면 파라미터 수가 너무 많은 것이 다층 퍼셉트론의 단점
  - MLP로 이미지를 다룰 때 공간적 특징의 손실이 발생함
    - 2차원 이미지를 1차원 벡터로 변형하면 이미지내 공간적 특징이 손실함
    - 그러면 서로 가까이 위치한 픽셀 간의 관계를 알 수 없기 때문에 정보의 손실이 발생함
    - 공간적 특징을 해결할 수 있는 방법이 Convolutional Neural Network가 됨

합성곱 신경망

- Convolution Neural Network이란
  - 2차원 행렬인 이미지를 1차원 벡터로 변환되면서 손실되는 정보와 Fully-connected layer의 계산 복잡도를 고려하여 이미지 입력을 다루기 위해 CNN이 탄생함
  - CNN은 2차원 이미지 행렬을 그대로 입력을 받을 수 있으므로 픽셀 값에 숨어 있는 패턴을 이해할 수 있음

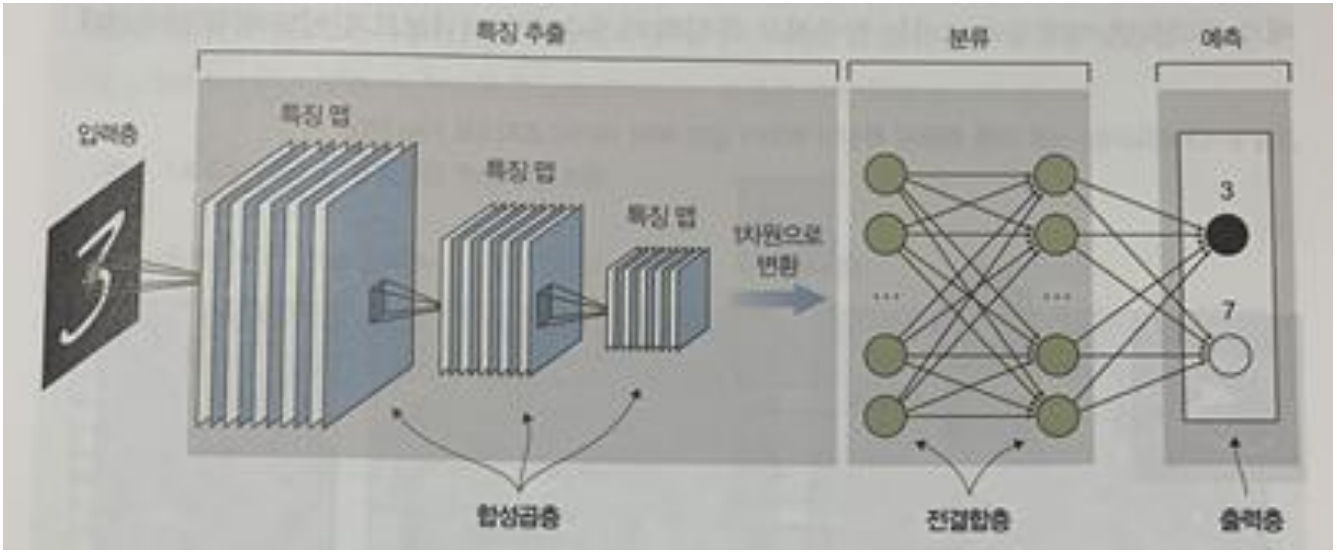
합성곱 신경망

- Convolution Neural Network구조
  - 첫번째 층에서는 기본적인 특징 (모서리, 직선)을 학습하고 그 다음 층에 서는 이보다 좀 더 복잡한 패턴(원, 정사각형 등)을 이어지는 층에서는 훨씬 복잡한 패턴(얼굴의 일부, 자동차 바퀴, 개의 수염 등)을 학습함
  - 추출한 특징이 Flatten을 통해 1차원 벡터로 만들어 지고 추출된 특징이 Fully connect layer에 들어가서 분류를 수행함 (위 과정은 앞서 다층 퍼셉트론 진행한 것과 유사함)



합성곱 신경망

- Convolution Neural Network구조
  - 입력층 / 특징 추출을 담당하는 Convolution layer / 분류를 담당하는 Fully connected layer / 예측 결과 출력
  - CNN은 특징 추출 / 분류 / 예측의 구조를 갖음



합성곱 신경망

- Convolution Neural Network구조
  - 이미지를 숫자 3과 7의 두개의 클래스로 분류하는 단계를 순서대로 표현해 설명함
    - 1. 이미지를 Convolutional layer (합성 곱)에 입력함
    - 2. 이미지가 합성 곱 층을 지나면 발견된 패턴이 Feature map (특징 맵)으로 추출됨, 합성 곱 층의 출력은 1차원 특징 벡터로 변환됨, 이 때 특징 추출이 끝날 때까지 이미지의 크기가 각 층을 지날 때마다 줄어들고, 특징 맵 수는 늘어나는 것을 볼 수 있음.
    - 1차원으로 변환된 특징 벡터는 이어지는 전결합층에 입력되어 이미지를 분류함
    - 예측 결과에 해당하는 출력층의 노드가 발화함, 여기서는 두가지 클래스 중 하나를 선택 (3과 7) 함

합성곱 신경망

- Convolution Neural Network구조
  - 특징 추출 과정 살펴보기
    - 특징 추출 단계는 설명하면 큰 이미지르 여러 개의 작은 특징 맵으로 나눈 뒤 이를 쌓아 벡터로 만드는 과정
    - 예를 들어, 숫자 3이 쓰인 이미지가 입력되었다면 (깊이 = 1) 이 이미지는 숫자 3을 나타내는 다양한 특징의 위치가 담긴 여러 개의 이미지로 조각조각 나눠짐
    - 특징 가지 수를 네가지라고 하면 깊이는 4가 됨
    - 이미지가 합성 곱층을 차례대로 통과하면 이미지의 크기는 작아지고 특징의 가짓수가 늘어나므로 깊이도 점점 깊어짐
    - **CNN에서는 실제로 이미지를 여러 조각으로 분할하지는 않고 실제로는 다른 사물이 찍힌 이미지와 구분되는 의미있는 특징을 추출하여 다른 특징과 함께 쌓아 둠**

합성곱 신경망

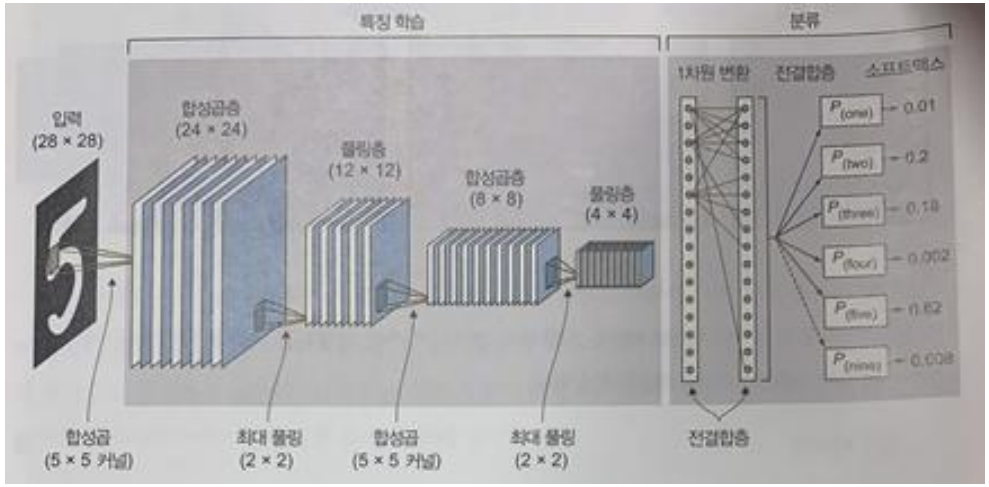
- Convolution Neural Network구조
  - 분류 과정 살펴보기
    - 특징 추출이 끝난 뒤에는 Fully Connected Layer (전결합층)을 추가해서 특징 벡터를 입력 받도록 함
    - 이 Fully Connected Layer는 첫번쨰 특징을 보면 모서리 같은 부분이 있어서 3,7 또는 2일 것임 두 번째 특징을 보면 곡선이 있는 것으로 보아하니 7은 확실이 아님 과 같은 형식으로 숫자 3이라는 결론에 이르게 됨



Convolutional Neural Network에서 추출된 Feature map 흐름

합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성곱층 (convolution layer)
  - 풀링 층 (pooling layer, PooL)
  - 전결합 층 (Fully connected layer)
  - 합성곱 신경망의 기본 구성 요소는 합성 곱 층과 전결합 층으로 구분함, 합성 곱층은 특징 추출, 전결 합 층은 분류를 수행할 수 있음

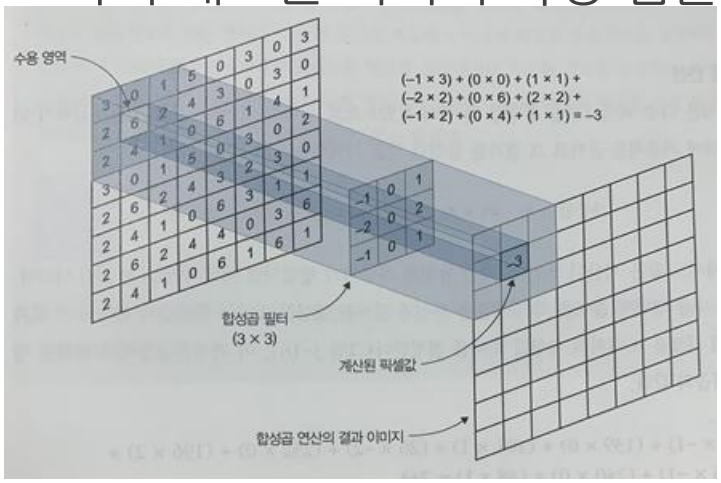


합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱층 (convolution layer)
    - Convolution layer는 CNN의 핵심 구성요소
    - 이미지 위를 픽셀 단위로 오가며 특징을 찾는 확대경과 같은 역할을 수행하며 이미지에 찍힌 사물을 구분 짓는 특징을 추출함
    - 합성 곱이란
      - 수학에서 말하는 합성 곱은 두 함수를 인수로 새로운 함수를 만들어 내는 연산임
      - CNN 신경망에서 말하는 합성 곱은 첫번째 인수는 입력 이미지, 두번째 인수는 합성 곱 필터에 해당함

합성곱 신경망

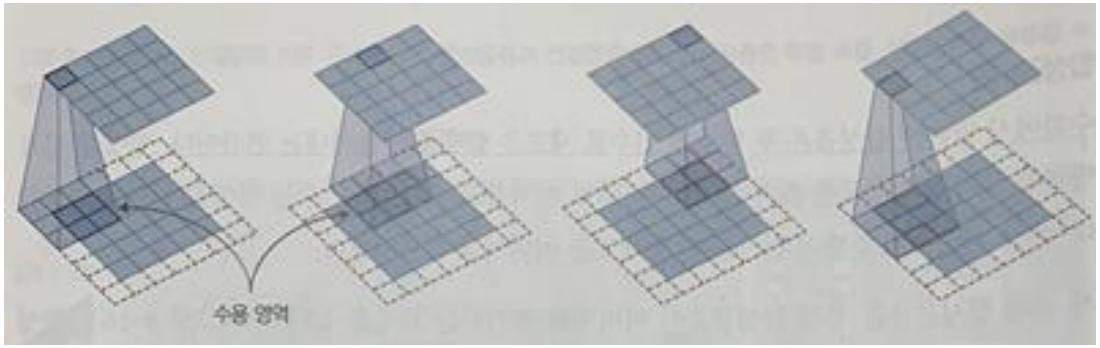
- Convolution Neural Network의 기본 요소
  - 합성 곱층 (convolution layer)
    - 첫 번째 합성 곱 층을 통해 합성 곱 층이 이미지를 처리하는 과정에 대해서 설명하고자 함
      - 합성 곱 필터를 입력 이미지 위로 이동시키며 합성 곱 필터가 위치한 부분에 해당하는 작은 이미지 조각을 처리한 결과를 모아서 새로운 이미지 특징 맵을 만듦



합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱층 (convolution layer)
    - 가운데 위치한 3x3크기의 행렬이 합성곱 필터 = 커널(kernel)
    - 커널은 입력 이미지 위를 픽셀 단위로 움직이며 연산을 수행하여 픽셀 값을 계산함, 그리고 각 위치에서 연산된 픽셀 값을 모아 '합성곱 연산을 거친' 새로운 이미지를 만들어 다음 층으로 전달함, 필터가 위치한 입력 이미지 상의 범위를 **수용 영역 (receptive field)**

- 커널은 입력 이미지 위를 픽셀 단위로 움직이며 연산을 수행하고, 합성곱 연산을 거치 새로운 이미지를 만들어 다음 층으로 전달함, 이때 합성곱 연산이 한 번 수행되는 입력 이미지 상의 범위를 수용영역이라고 함



합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱층 (convolution layer)
    - 여기서 커널 값은 무엇을 가리는가?
    - 합성 곱 신경망에서는 합성 곱 행렬이 바로 가중치임
    - 합성 곱 행렬을 무작위 값으로 초기화(randomly initialized) 되며 신경망에 의해 학습이 됨
    - 수용 영역과 합성곱 필터의 겹치는 픽셀 값을 각각 곱하고 그 결과를 합해서 필터 중심에 해당하는 픽셀의 픽셀 값을 계산해 새로운 이미지를 만듦



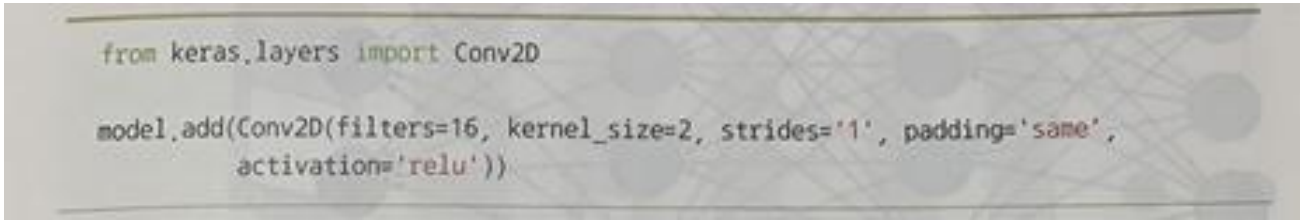
합성곱 연산 예시

합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱층 (convolution layer)
    - 합성 곱 필터 (커널)는 입력 이미지 위를 이동하며 입력 이미지 전체를 커버함
    - 합성 곱 필터가 한 번 움직일 때마다 위와 같은 픽셀 단위로 가중합이 계산되어 필터 중심에 해당하는 픽셀의 새로운 값이 결정됨
    - 이런 방법으로 만들어진 이미지를 특징 맵 (feature map) 또는 활성화 맵 (activation map)

합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱층 (convolution layer)
    - 합성 곱층에는 하나 또는 그 이상의 합성곱 필터가 있음
    - 이전 합성곱층의 합성곱 필터 수만큼 특징 맵(합성 곱 연산으로 생성된 이미지)이 출력 되기 때문에 이 필터 수가 다음 층의 깊이를 결정함
    - 케라스에서 구현 내용
      - Filters : 합성곱 필터 수, 출력의 깊이를 결정 / Kernel size : 합성곱 필터 행렬(커널)의 크기 / Strides : 스트라이드 / Padding : 패딩

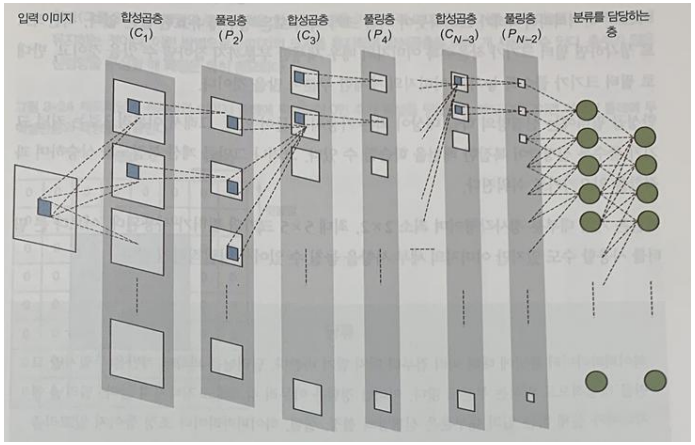


```
from keras.layers import Conv2D

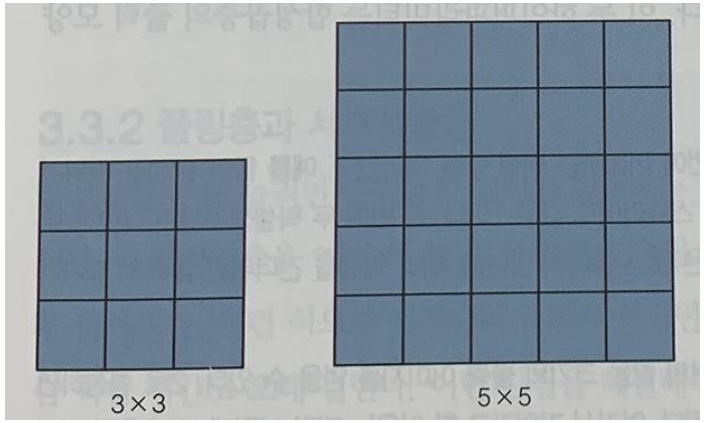
model.add(Conv2D(filters=16, kernel_size=2, strides='1', padding='same',
          activation='relu'))
```

합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱층 (convolution layer)
    - 합성 곱 필터 수 :
      - 합성 곱 필터를 커널이라고 함, 커널은 가중치가 담긴 행렬로, 입력 이미지를 위를 이동하면서 특징을 추출하는 역할, 커널 크기는 합성 곱 필터 크기 (너비x높이)를 의미함



- 커널수를 함께 나타낸 CNN의 구조



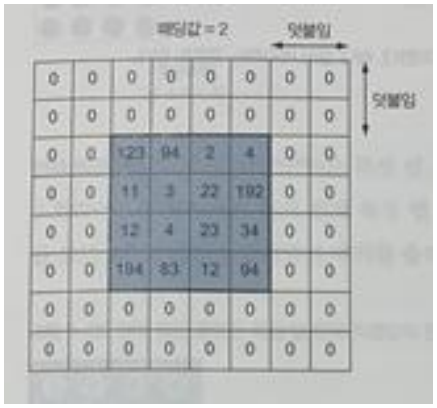
- 커널의 크기 예시

합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱 필터 수 :
    - 커널 크기는 합성 곱층을 만들기 위해 필요한 파라미터 중 하나
    - 직관적으로 필터의 크기가 작으면 작을 수록 이미지의 세세한 부분까지 잡아 낼 수 있을 것이고, 반대로 필터 크기가 클수록 놓치는 이미지의 세세한 부분이 많이 있을 것임
  - 스트라이드와 패딩:
    - 스트라이드 (stride) : 필터가 입력 이미지 위를 한 번에 이동하는 픽셀 수를 의미, 예를 들어 합성곱 필터가 입력 이미지 위를 한번에 한 픽셀씩 이동한다면 스트라이드 값은 1, 한 번에 두 픽셀씩 이동한다면 스트라이드 값은 2

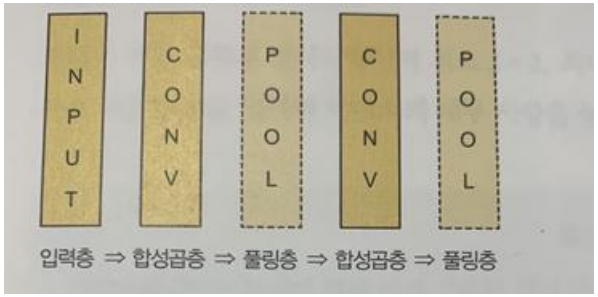
합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 스트라이드와 패딩 (계속):
    - 패딩 (padding) : 흔히 말하는 제로패딩 (zero-padding)은 이미지 둘레에 픽셀 값이 0인 추가 픽셀을 덧붙이는 것을 말함
    - 패딩의 주 목적은 합성곱 연산을 거친 출력 이미지의 크기를 입력 이미지의 크기와 같게 유지하는 것
    - 제로 패딩을 적용하면 이미지 둘레에 픽셀 값이 0인 추가 픽셀을 덧붙임, 패딩 값이 2이면 이미지 둘레에 두 픽셀 만큼의 패딩을 덧붙임



합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 풀링 층과 서브 샘플링 :
    - 풀링 (pooling) 은 다음 층으로 전달되는 파라미터 수를 감소시키는 방법, 풀링 연산은 최대 또는 평균과 같은 통계 함수로 입력 크기를 축소해 다음 층으로 전달되는 파라미터 수를 감소함
    - 풀링층의 목적은 합성곱층에서 만든 특징 맵을 다운 샘플링해서 파라미터 수를 줄여 계산 복잡도는 낮추는 것임
    - CNN구조는 일반적으로 합성 곱층 사이에 풀링 층을 끼워 넣는 경우가 많음



합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 풀링 층과 서브 샘플링 :
    - 최대 풀링:
      - Max pooling : 자기 앞의 합성곱 층에서 출력한 특징 맵을 입력 받아 커널을 입력 이미지 위로 이동시키면서 윈도우 내 픽셀 값의 최대값을 찾아 이를 출력 이미지의 픽셀 값으로 삼음
      - 스트라이드가 2인 2by2크기의 풀링은 4x4크기의 특징 맵을 2x2로 축소함



- Max pooing 적용 예시

합성곱 신경망

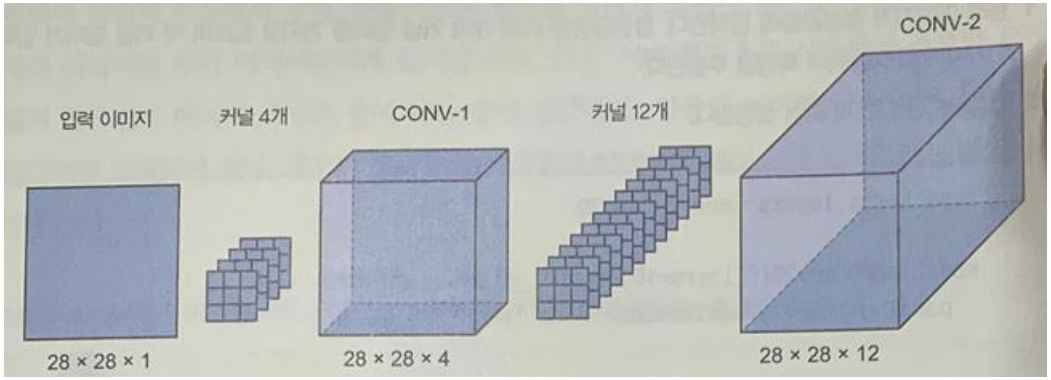
- Convolution Neural Network의 기본 요소
  - 풀링 층과 서브 샘플링 :
    - 풀링 층을 사용하는 이유 :
      - 신경망 전체의 파라미터 수가 어마어마하게 늘어남
      - 이때 풀링층을 사용하면 중요한 특징을 잃지 않으면서 이미지 크기를 줄여 다음 층에 전달이 가능하기 때문
    - 케라스에서 구현 :

```
from keras.layers import MaxPooling2D

model.add(MaxPooling2D(pool_size=(2, 2), strides = 2))
```

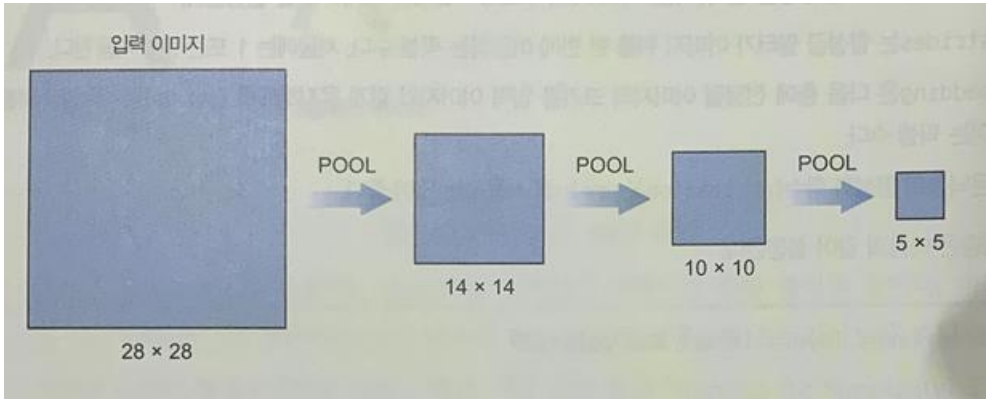
합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱 층의 입력과 출력
    - 28 by 28 인 입력 이미지가 필터 수 4, 스트라이드와 패딩이 1로 설정된 convolutional layer를 통과하면 출력 이미지의 크기는 입력 이미지와 동일하지만 이미지의 깊이는 4가 됨, 커널 12개틀 통과하면 이미지 깊이는 12가 됨



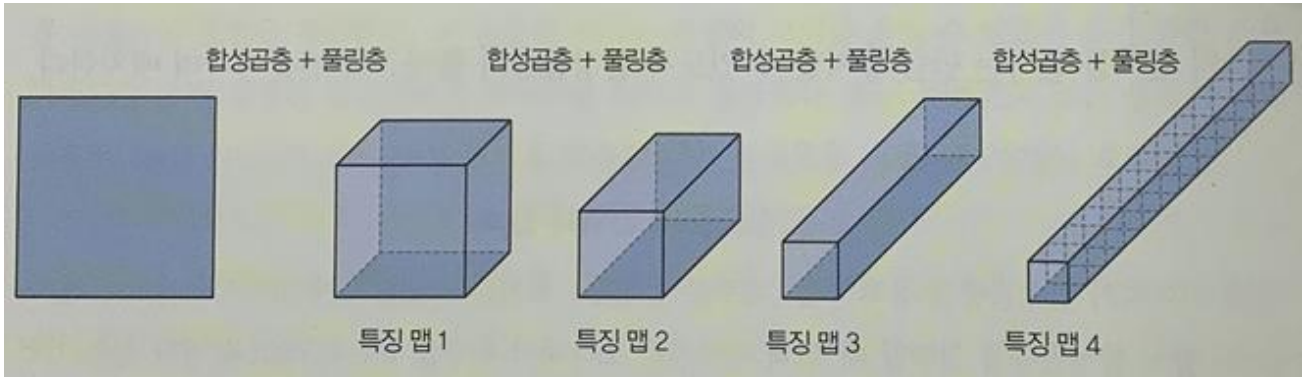
합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱 층의 입력과 출력 (계속)
    - 풀링층을 통과한 출력은 깊이는 그대로 유지되지만 가로세로 크기는 줄어듬



합성곱 신경망

- Convolution Neural Network의 기본 요소
  - 합성 곱 층의 입력과 출력 (계속)
    - 합성곱층과 풀링층을 번갈아 배치한 구성

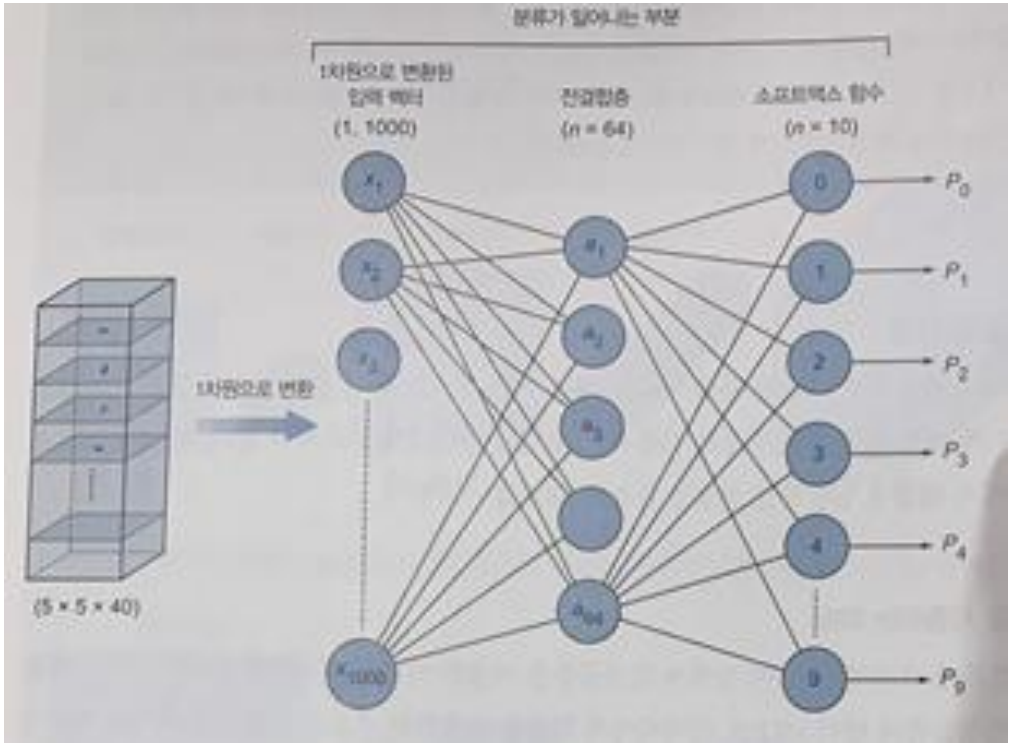


합성곱 신경망

- Convolution Neural Network의 기본 요소
  - Fully connected layer (FC Layer)
    - 추출된 특징을 사용하여 실제로 이미지를 분류해야 함
    - 이미지 분류를 위해 MLP를 사용함
    - FC-layer를 사용하는 이유
      - MLP는 분류에 효과적임
    - FC layer의 구성
      - 1차원 벡터 입력 : (1,n)모양의 1차원 벡터로 변환해 사용해야 함
      - 은닉층 : 하나 이상의 층수를 가짐
      - 출력층 : 분류문제를 위한 출력층의 활성화 함수는 소프트 맥스 함수를 사용해야 함

합성곱 신경망

- Convolution Neural Network의 기본 요소
  - Fully connected layer (FC Layer)

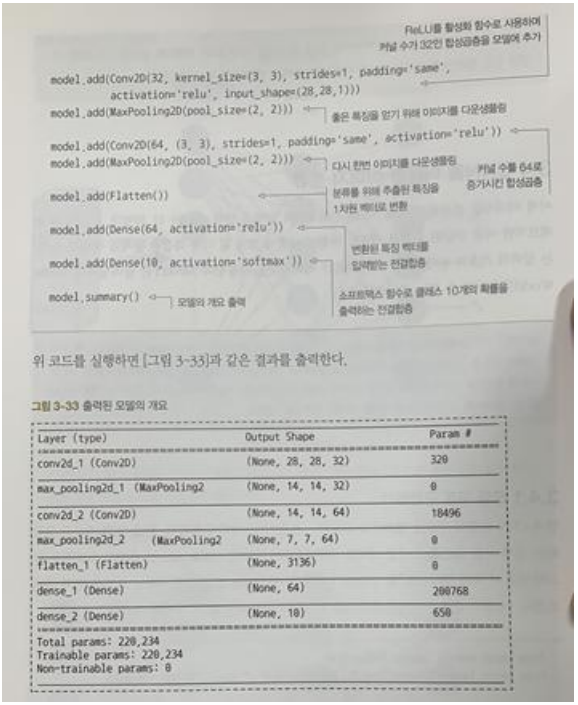


합성곱 신경망

- Convolution Neural Network을 이용한 이미지 분류
  - 모델 구조 구성하기

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout

model = Sequential()  <-- 모델 객체 생성
```



위 코드를 실행하면 [그림 3-33]과 같은 결과를 출력한다.

그림 3-33 출력된 모델의 개요

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 28, 28, 32)	320
max_pooling2d_1 (MaxPooling2	(None, 14, 14, 32)	0
conv2d_2 (Conv2D)	(None, 14, 14, 64)	18496
max_pooling2d_2 (MaxPooling2	(None, 7, 7, 64)	0
flatten_1 (Flatten)	(None, 3136)	0
dense_1 (Dense)	(None, 64)	200768
dense_2 (Dense)	(None, 10)	650

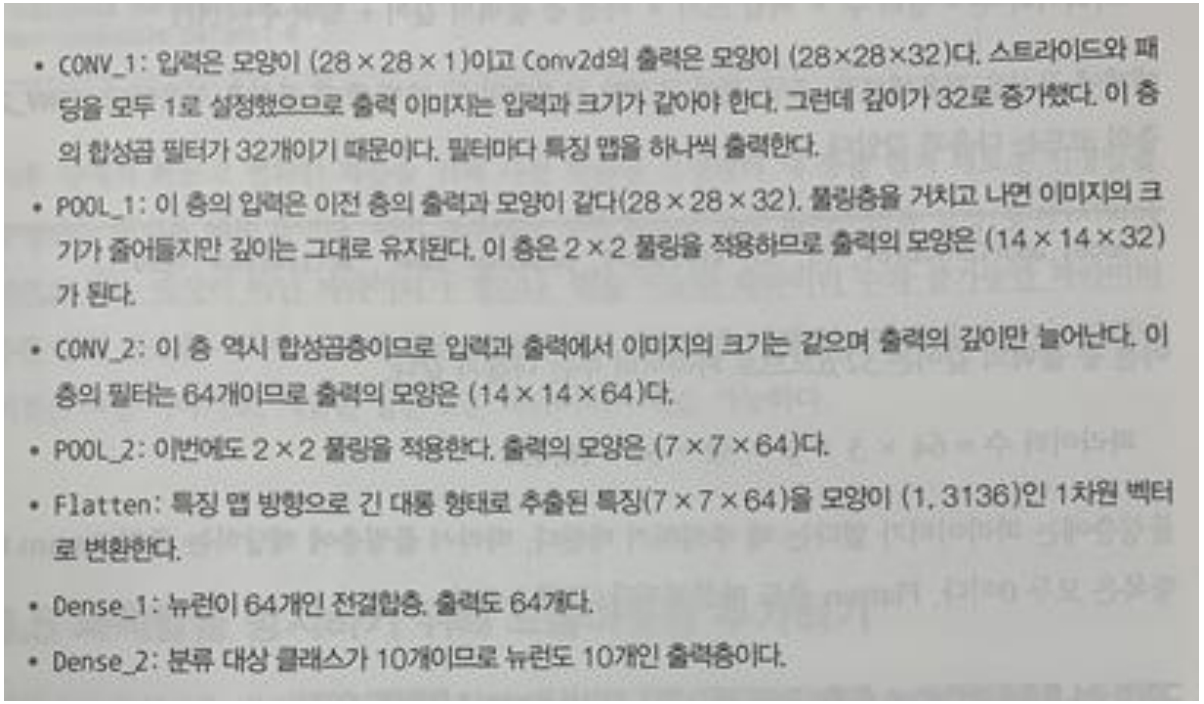
Total params: 220,234
Trainable params: 220,234
Non-trainable params: 0

합성곱 신경망

- Convolution Neural Network을 이용한 이미지 분류
  - 모델 구조 구성하기 세부 설명:
    - Input_shape 인수는 첫번 재 합성 곱층에만 지정, 그 다음 부터는 입력 모양을 지정해 줄 필요가 없음, 이전 층의 출력 모양이 이미 결정되어 있기 때문
    - 모든 합성 곱층과 풀링 층의 출력은 모양이 (None, height, width, channels)인 3차원 텐서, 먼저 height, width는 출력 이미지의 가로 세로 크기, channels는 출력 이미지의 깊이, 다시 말해 특징 맵 수, 튜플의 첫번째 요소인 None는 이 층에서 처리된 이미지 수, 이 값이 None이면 batch_size를 자유롭게 정할 수 있음
    - 출력된 모델의 개요에서 Output-shape항목을 보면 앞서 설명한 내용처럼 신경망의 층이 거듭될 수 록 출력되는 이미지의 크기가 작아지는 것을 확인할 수 있음

합성곱 신경망

- Convolution Neural Network을 이용한 이미지 분류
  - 모델 구조 구성하기 라인 별 설명:



- CONV_1: 입력은 모양이 (28 × 28 × 1)이고 Conv2d의 출력은 모양이 (28×28×32)다. 스트라이드와 패딩을 모두 1로 설정했으므로 출력 이미지는 입력과 크기가 같아야 한다. 그런데 깊이가 32로 증가했다. 이 층의 합성곱 필터가 32개이기 때문이다. 필터마다 특징 맵을 하나씩 출력한다.
- POOL_1: 이 층의 입력은 이전 층의 출력과 모양이 같다(28 × 28 × 32). 풀링층을 거치고 나면 이미지의 크기가 줄어들지만 깊이는 그대로 유지된다. 이 층은 2 × 2 풀링을 적용하므로 출력의 모양은 (14 × 14 × 32)가 된다.
- CONV_2: 이 층 역시 합성곱층이므로 입력과 출력에서 이미지의 크기는 같으며 출력의 깊이만 늘어난다. 이 층의 필터는 64개이므로 출력의 모양은 (14 × 14 × 64)다.
- POOL_2: 이번에도 2 × 2 풀링을 적용한다. 출력의 모양은 (7 × 7 × 64)다.
- Flatten: 특징 맵 방향으로 긴 대롱 형태로 추출된 특징(7 × 7 × 64)을 모양이 (1, 3136)인 1차원 벡터로 변환한다.
- Dense_1: 뉴런이 64개인 전결합층. 출력도 64개다.
- Dense_2: 분류 대상 클래스가 10개이므로 뉴런도 10개인 출력층이다.

질문 및 정리

